O M N I S E N S E

Emergency Vehicle Rerouting System

Traffic Simulation & Rerouting Validation — Technical Document

1. Simulation Module Overview

■ SCOPE & OBJECTIVES

This document details the complete technical requirements for the Simulation subsystem of Omnisense — covering the tools, libraries, datasets, processing pipeline, and validation criteria needed to build, run, and test emergency vehicle rerouting logic in a simulated traffic environment.

Core Approach

The simulation module acts as the testing and validation backbone of Omnisense. Before deploying rerouting decisions to real traffic signal hardware, every scenario is first modelled and stress-tested in SUMO, using real road data downloaded via OSMnx and pathfinding algorithms from NetworkX.

The simulation module covers four functional areas:

- Road Network Modelling — download and build real-world road graphs from OpenStreetMap using OSMnx, supporting city-scale intersections and lane data.
- Traffic Flow Simulation — run microscopic traffic simulations with SUMO to model realistic vehicle movement, signal timings, and congestion patterns.
- Emergency Rerouting Tests — inject emergency vehicle events and validate that rerouting decisions clear a path within target response times.
- Signal Control Integration — use SUMO's TraCI API to dynamically override traffic signal states, testing how rerouting commands behave against live signal controllers.

2. Core Simulation Libraries

■ PYTHON & EXTERNAL TOOLS REQUIRED

All libraries listed below are 100% free and open-source. The complete simulation stack runs on Python 3.9+.

Library	Tag	Install	Purpose

SUMO + TraCI	SIMULATION	`sudo apt install sumo`	Microscopic traffic simulator. TraCI API enables real-time Python control of signals and vehicles during simulation runs.
OSMnx	ROAD NET.	`pip install osmnx`	Downloads real-world road networks from OpenStreetMap. Generates graph objects and SUMO-compatible network XML files.
NetworkX	GRAPH ALGO	`pip install networkx`	Graph analysis and pathfinding (Dijkstra, A*). Computes optimal emergency vehicle routes on the road network.
GeoPy / Shapely	GEOSPATIAL	`pip install shapely geopy`	Geospatial proximity calculations. Identifies intersection zones and detects EV entry into clearance radius.
Pandas / NumPy	DATA PROC.	`pip install pandas numpy`	Timestamped sensor stream ingestion and feature engineering. Processes GPS coordinates and speed data.

3. Simulation Processing Pipeline

■ STEP-BY-STEP DATA & CONTROL FLOW

The following 8-step pipeline describes the complete data and control flow from road network acquisition through to logged validation results.

#	Stage	Description	Tool
01	**OSM Download**	OSMnx fetches the drivable road graph for the target city from OpenStreetMap, returning a directed multigraph with junction, lane, and speed-limit data.	`osmnx`
02	**Network Build**	The OSM graph is converted to a SUMO-compatible network XML file using ox.save_graph_xml(). This defines all edges, junctions, and lane connections.	`osmnx / sumo`
03	**SUMO Init**	SUMO loads the network file alongside a traffic demand scenario generated by randomTrips.py. The simulation is initialised but not yet stepped forward.	`sumo`
04	**TraCI Control**	A Python process connects to SUMO via the TraCI API. Every simulation tick is now driven by Python, enabling real-time control of signals and vehicles.	`traci`
05	**EV Injection**	An emergency vehicle is added to the simulation at a chosen origin via traci.vehicle.add(). Its route and speed profile replicate a real EV dispatch event.	`traci`
06	**Rerouting**	NetworkX runs Dijkstra or A* over the road graph to compute the optimal path from the EV's current position to the destination, weighted by travel time.	`networkx`

07	**Signal Override**	TraCI commands clear intersections along the computed route before the EV arrives. MQTT also dispatches these commands to physical signal controllers.	`traci / mqtt`
08	**Validation**	Pandas logs every timestep: EV travel time, signal clearance delta, civilian delay, and path optimality. Results are exported as CSV for analysis.	`pandas`

4. SUMO + TraCI — Deep Dive

4.1 SUMO Capabilities

■ TRAFFIC SIMULATION ENGINE

- Microscopic simulation of individual vehicles with realistic car-following models
- Realistic traffic signal phase modelling with configurable phase durations
- Multi-lane road network support with lane-change behaviour
- Supports 1,000+ vehicles simultaneously in real-time
- Import OSMnx-generated network XML directly — no manual conversion needed
- Outputs detailed trip logs: speed, wait time, and fuel consumption per vehicle
- GUI mode for visual debugging + headless (CLI) mode for batch automated runs

4.2 TraCI Python API — Key Commands

■ REAL-TIME SIMULATION CONTROL

TraCI (Traffic Control Interface) is the Python API that allows the Omnisense fusion engine to interact with SUMO in real-time. Below are the critical commands used in the simulation:

```
import traci

# Start simulation with config file
traci.start(['sumo', '-c', 'bengaluru.sumocfg'])

# Inject emergency vehicle at origin
traci.vehicle.add('EV_001', routeID='route_hospital', typeID='emergency')
traci.vehicle.setSpeed('EV_001', speed=25.0)

# Override traffic signal at junction to green for EV lane
traci.trafficlight.setPhase('junction_42', phase=2)

# Query live congestion data from edge
travel_time = traci.edge.getTraveltime('edge_main_road')

# Advance simulation by one step (1 second per tick)
traci.simulationStep()
```

```
# Get EV position for proximity detection
x, y = traci.vehicle.getPosition('EV_001')
```

5. OSMnx + NetworkX — Road Network & Pathfinding

5.1 OSMnx — Network Retrieval

■ **OPENSTREETMAP ROAD GRAPH DOWNLOAD**

OSMnx downloads the complete drivable road network for any city and converts it into a Python NetworkX graph object, which is then exported as SUMO-compatible XML for simulation use.

```
import osmnx as ox

# Download drivable road graph for Bengaluru
G = ox.graph_from_place('Bengaluru, India', network_type='drive')

# Export for SUMO ingestion as network XML
ox.save_graph_xml(G, filepath='bengaluru.osm')

# Get node / edge GeoDataFrames for geospatial analysis
nodes, edges = ox.graph_to_gdfs(G)

# Find nearest graph node to a GPS coordinate
origin_node = ox.distance.nearest_nodes(G, X=77.5946, Y=12.9716)
```

5.2 NetworkX — Emergency Pathfinding

■ **DIJKSTRA / A* SHORTEST PATH ROUTING**

NetworkX provides the graph algorithms to compute the shortest emergency vehicle path. Two strategies are supported depending on computational constraints:

```
import networkx as nx

# Strategy 1: Dijkstra — guaranteed optimal, suitable for smaller graphs
path = nx.shortest_path(
    G, source=origin_node, target=hospital_node, weight='travel_time'
)

# Strategy 2: A* — faster for large city-scale graphs with distance heuristic
def dist_heuristic(u, v, data):
    return ox.distance.great_circle_vec(
        G.nodes[u]['y'], G.nodes[u]['x'],
        G.nodes[v]['y'], G.nodes[v]['x']
    )
```

```
path = nx.astar_path(
    G, origin_node, hospital_node,
    heuristic=dist_heuristic, weight='length'
)
```

6. Simulation Datasets

■ **FREE & OPEN-SOURCE DATA SOURCES**

All datasets used in the simulation module are freely available without registration or licensing fees.

6.1 OpenStreetMap (Road Network)

Source: openstreetmap.org — accessed via OSMnx Python library

- Complete real-world road network data for any city worldwide
- Junction and lane topology including turn restrictions and signal locations
- Speed limits, road types, and one-way designations included
- Exported as SUMO-compatible .osm network XML with no manual editing required

6.2 SUMO Built-in Scenario Generator (Traffic Flow)

Source: eclipse.dev/sumo — ships with the SUMO installation

- randomTrips.py generates realistic vehicle demand from road network alone — no external dataset required
- duarouter computes dynamic user-assignment routes from origin-destination demand matrices
- Traffic volumes can be calibrated against real-world loop detector counts if available

6.3 GPS / Sensor Logs via Pandas (EV Telemetry)

Source: simulated via TraCI, or real IoT devices in production deployment

- Timestamped GPS coordinates and speed readings ingested via Pandas DataFrames
- In simulation, TraCI provides these values directly each tick via traci.vehicle.getPosition()
- In production, the same Pandas pipeline reads from real GPS sensors streamed over MQTT

7. KPIs & Validation

■ **WHAT TO MEASURE & PROVE**

All simulation runs are logged via Pandas and evaluated against the following performance targets before the system is considered ready for production deployment.

KPI	Target	How to Measure
Signal Clearance Time	**< 30 seconds**	Time from EV detection to intersection cleared, measured per junction across all test scenarios
Rerouting Success Rate	**≥ 95%**	Percentage of simulation runs where a valid path is found and executed without deadlock or collision
TraCI Command Latency	**< 5 ms**	Round-trip time for each Python-to-SUMO control command (signal override, route update) per simulation step
EV Travel Time Delta	**< −20%**	Reduction in EV travel time vs. baseline (no rerouting active), averaged across all test scenarios
Civilian Delay Introduced	**< 10 seconds**	Average additional wait time imposed on civilian vehicles due to signal overrides — minimise collateral impact

7.1 Validation Test Cases

- Single EV dispatch at peak traffic hour — standard load scenario
- Multiple EVs on converging routes — tests priority and conflict resolution
- Signal failure / fallback routing — resilience when a signal controller is unresponsive
- Road closure with dynamic reroute mid-journey — tests live path recalculation
- EV vs. opposing traffic priority — ensures civilian safety alongside clearance

8. Simulation Stack Summary

The table below provides a consolidated reference of all five simulation-specific tools, their roles, and install commands.

Tool	Role in Omnisense Simulation	Install
SUMO + TraCI	Core simulation engine + real-time Python signal control	`sudo apt install sumo`
OSMnx	Real road network graph download from OpenStreetMap	`pip install osmnx`
NetworkX	Dijkstra / A* emergency vehicle pathfinding	`pip install networkx`
GeoPy / Shapely	Geospatial proximity detection and zone identification	`pip install shapely geopy`
Pandas / NumPy	Sensor stream processing, KPI logging, CSV export	`pip install pandas numpy`

All tools are 100% free and open-source · Python 3.9+ · GPU training: pip install torch --index-url https://download.pytorch.org/whl/cu118